Ayame: Fast Logging for Serial Safety Net

Shun Sasaki[1,a)] Takayuki Tanabe[1,b)] Hideyuki Kawashima[2,c)]

Abstract: Concurrency control is essential for database access, and the Serial Safety Net (SSN) represents one of the state-of-the-art methods in this field. Integrating SSN with logging ensures that transaction processing is recoverable. While P-WAL is an efficient logging method that leverages parallel I/O extensively, it still suffers from certain limitations. To overcome these drawbacks, this paper proposes a novel logging method, Ayame. The proposed method incorporates three optimization techniques: the elimination of global counters, dependency analysis, and asynchronous I/O. Experimental results demonstrate that Ayame achieves up to a 4.5-fold performance improvement over P-WAL in YCSB-A.

1. Introduction

1.1 Motivation

Transaction processing is a core abstraction behind data-intensive applications such as banking, payment processing, inventory management, ticket reservation, and online services that update shared state under high concurrency. A transaction processing system must provide both correct concurrent execution and crash recovery; in this sense, transaction processing consists of concurrency control (CC) and recovery [1]. A large body of work has studied scalable CC for multicore main-memory databases, including Silo, ERMIA, TicToc, SI, SSI, and SSN-style protocols [2], [3], [4], [5], [6], [7]. These protocols improve isolation and scalability by reducing centralized validation, timestamp, and version-management bottlenecks. In contrast, the recovery side of modern CC protocols has been explored less systematically: when a high-performance CC protocol is combined with crash durability, it is often unclear which recovery protocol preserves both safety and scalability.

Write-ahead logging (WAL) is the standard foundation for database recovery, with ARIES being the classical design [8], [9]. Modern systems have revisited WAL for multicore and fast-storage environments. SiloR parallelizes logging, checkpointing, and recovery for Silo; Passive Group Commit reduces synchronization overhead on non-volatile memory; and P-WAL partitions the log into multiple streams [10], [11], [12]. Although these systems differ in the target storage device, log format, and recovery procedure, they share an important lesson: writing log records in parallel is essential for making durability compatible with multicore transaction processing. At the same time, strong multiversion protocols such as SI, SSI, and SSN introduce timestamp and dependency structures that are not fully reflected in conventional parallel logging designs. The motivation of this work is therefore to clarify how P-WAL-style parallel logging should be combined with such timestamp-based CC protocols, especially SSN, without unnecessarily reintroducing global ordering and durable-prefix bottlenecks.

1.2 Problem

We target SSN because it is the most complex of the SI-family protocols we consider: it certifies serializability using dependency metadata, while the underlying engine still uses timestamped multiversion execution. As the baseline recovery protocol, we start from P-WAL, which is also used as a foundation of Shirakami, the transaction processing mechanism of Tsurugi [12], [13]. P-WAL is attractive because it removes the single-WAL insertion bottleneck by assigning transactions to multiple log streams. However, combining P-WAL with SSN exposes three problems.

P1. Multiple Global Counter Accesses: First, P-WAL can introduce frequent global-counter accesses. Even if the CC layer already assigns a commit timestamp for SSN validation and serialization, the recovery layer may allocate a separate global LSN or maintain a global durable prefix. This duplicates the ordering work already performed by the CC layer and adds shared-memory synchronization to the commit path.

P2. Waiting for Data Transfer: Second, workers can spend a long time waiting around `fdatasync`. P-WAL parallelizes log streams, but a transaction cannot be acknowledged until the corresponding durable condition is satisfied. If worker threads wait directly for log flushes or durable-prefix updates, storage synchronization and notification latency reduce the effective scalability of the CC protocol.

[1] Graduate School of Media and Governance, Keio University, Japan
[2] Faculty of Environment and Information Studies, Keio University, Japan
[a)] shun.sasaki@keio.jp
[b)] tanab@keio.jp
[c)] river@sfc.keio.ac.jp

© 1992 Information Processing Society of Japan

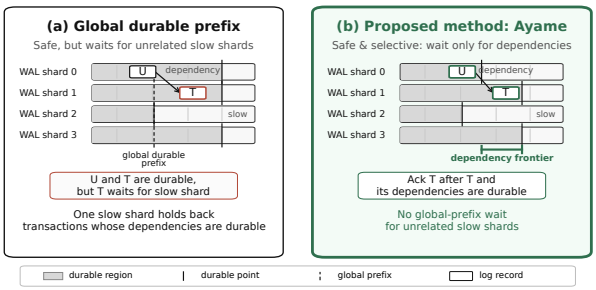

Fig. 1 Durable acknowledgment policies in parallel WAL. Global-prefix acknowledgment is safe but can delay transactions whose dependencies are already durable because it waits for unrelated slow shards. Local-only acknowledgment avoids this delay but is unsafe. Ayame uses dependency-closed acknowledgment, acknowledging a transaction only after its own log record and dependency frontier are durable.

P3. Total History: Third, global-prefix acknowledgment waits for log records that are unrelated to the transaction being acknowledged. This rule is safe, but overly conservative: if one WAL shard is slow, transactions on other shards may be delayed even when they have no dependency on transactions logged by the slow shard. The opposite design point, local-only acknowledgment, avoids this delay but is unsafe. If transaction T reads a value written by transaction U, acknowledging T before U is recoverable may allow a crash recovery state that contains T without the state on which T depends.

Figure 1 summarizes these alternatives. Global-prefix acknowledgment is safe but may wait for unrelated slow shards. Local-only acknowledgment avoids such waits but can acknowledge a transaction before its dependencies are recoverable. The desired point is dependency-closed acknowledgment: acknowledge a transaction after its own log record and the transactions it depends on are durable.

In asynchronous durability protocols, logical commits and durable commit acknowledgments must be distinguished. A worker may continue after enqueueing a log record, but the transaction is not durable from the client's perspective until the system returns a durable acknowledgment. Therefore, this paper uses *durable-acknowledgment throughput* as the main throughput metric and reports it together with pending durable commits and tail latency.

1.3 Approach

This paper proposes *Ayame*, a timestamp-integrated, dependency-closed parallel WAL protocol for main-memory transaction processing. Ayame targets timestamp-based engines such as ERMIA, where the transaction engine already computes a logical commit order. Its goal is not merely to maximize raw logical commit throughput, but to return durable commit acknowledgments safely without unnecessary global-prefix waiting.

Ayame addresses the three problems above with three corresponding techniques.

A1. Commit timestamps as logical LSNs. Ayame reuses the SSN/ERMIA commit timestamp as the logical LSN for recovery. WAL shards still maintain local physical positions, but the WAL layer does not allocate an additional global logical order. This removes duplicated global ordering between CC and recovery and eliminates WAL-side separate global LSN allocation on the commit path.

A2. Worker/flusher/committer separation. Ayame decouples transaction execution from durable acknowledgment. Workers execute and validate transactions, assign commit timestamps, and enqueue log records. Flushers batch and persist shard-local logs. Committers observe durable-frontier advances and return durable acknowledgments. This pipeline keeps workers from directly blocking on `fdatasync` or on durable-prefix notification waits.

A3. Dependency-closed durable acknowledgment. Ayame replaces global-prefix acknowledgment with dependency-closed acknowledgment. For each transaction, Ayame maintains a dependency frontier that summarizes the log positions that must be recoverable before the transaction can be acknowledged. A transaction is acknowledged only when its own log record and this dependency frontier are durable. This preserves recovery safety while avoiding waits for unrelated WAL shards.

We implement Ayame in an ERMIA-style engine and evaluate it against P-WAL using SSN and YCSB workloads. The detailed experimental methodology, figures, and tables are presented in Section 4.

1.4 Organization

The rest of this paper is organized as follows. Section 2 describes transaction processing, including serial-safety-net and parallel logging protocols. Section 3 presents Ayame, which is a fast logging protocol. Section 4 describes the evaluation of Ayame compared with the P-WAL protocol using SSN. Section 5 discusses related work. Section 6 finally concludes this paper.

2. Preliminaries

2.1 Concurrency Control
2.2 Timestamp-based Protocols
2.2.1 Serial Safety Net
2.3 Write Ahead Logging for Recovery
2.4 Logging
2.4.1 P-WAL

P-WAL is attractive because it removes the single-WAL insertion bottleneck by assigning transactions to multiple log streams. However, combining P-WAL with SSN exposes three problems.

P1. Multiple Global Counter Accesses: First, P-WAL can introduce frequent global-counter accesses. Even if the CC layer already assigns a commit timestamp for SSN validation and serialization, the recovery layer may allocate a separate global LSN or maintain a global durable prefix. This duplicates the ordering work already performed by the CC layer and adds shared-memory synchronization to the commit path.

P2. Waiting for Data Transfer: Second, workers can

© 1992 Information Processing Society of Japan

Algorithm 1: Dependency Collection

```
1  Input: transaction T and accessed version v
2  procedure ReadVersion(T, v)
3    T.dep ← max(T.dep, v.write_frontier)
4    Add v to T's read set.
5  procedure OverwriteVersion(T, v)
6    T.dep ← max(T.dep, v.write_frontier)
7    T.dep ← max(T.dep, v.read_frontier)
8    Add v to T's write set.
```

Fig. 2 Dependency collection during transaction execution. A transaction inherits the frontier of versions it reads and conservatively inherits both writer and reader frontiers when overwriting a version.

Algorithm 2: Transaction Commit

```
1  Input: transaction T with dependency frontier T.dep
2  if ValidateAndSSNCheck(T) fails then abort T.
3  if T is read-only then
4    if T.dep is empty or already durable then
5      Ack(T) and return.
6    RegisterReadOnlyRequest(T.dep) and return.
7  T.c ← ERMIA/SSN commit timestamp.
8  T.log ← ChooseWalShard(worker_id).
9  record ← BuildWalRecord(T.c, T.dep, T.write_set).
10 T.seq ← Append(wal[T.log], record).
11 T.closed ← T.dep.
12 T.closed[T.log] ← max(T.closed[T.log], T.seq).
13 Publish T.closed to versions written by T.
14 Publish T.closed to read frontiers of versions read by
   read-write T.
15 RegisterCommitRequest(T.log, T.seq, T.dep, T.c).
16 Return logical commit to the worker.
```

Fig. 3 Ayame commit protocol. Workers do not wait for `fdatasync`; they append a WAL record, publish the closed frontier, and register a durable-ack request. Read-only transactions do not append WAL records, but they still wait if they observed non-durable dependencies.

spend a long time waiting around `fdatasync`. P-WAL parallelizes log streams, but a transaction cannot be acknowledged until the corresponding durable condition is satisfied. If worker threads wait directly for log flushes or durable-prefix updates, storage synchronization and notification latency reduce the effective scalability of the CC protocol.

P3. Total History: Third, global-prefix acknowledgment waits for log records that are unrelated to the transaction being acknowledged. This rule is safe, but overly conservative: if one WAL shard is slow, transactions on other shards may be delayed even when they have no dependency on transactions logged by the slow shard. The opposite design point, local-only acknowledgment, avoids this delay but is unsafe. If transaction T reads a value written by transaction U, acknowledging T before U is recoverable may allow a crash recovery state that contains T without the state on which T depends.

3. Proposal: Ayame

4. Evaluation

We implement Ayame in an ERMIA-style engine and evaluate it using YCSB workloads with the transaction worker count on the x-axis. The comparison uses the same ERMIA-PWAL binary for all WAL systems; Single WAL and P-WAL are selected by runtime WAL-mode flags, while Ayame additionally runs background flusher/logger and committer threads. At 32 worker threads on YCSB-A, Single WAL achieves 10K durable acknowledgments per second and P-WAL achieves 66K, whereas Ayame achieves 298K durable acknowledgments per second. On YCSB-B, Single WAL achieves 28K and P-WAL achieves 238K, whereas Ayame achieves 553K durable acknowledgments per second. These results correspond to 4.5× and 2.3× improvements over P-WAL on YCSB-A and YCSB-B, respectively. Ayame also keeps the number of pending durable commits small at 32 worker threads: 703 pending commits on YCSB-A and 58 pending commits on YCSB-B.

Algorithm 3: Durable Acknowledgment

```
1  procedure RegisterCommitRequest(log, seq, dep, c)
2    req ← dep.
3    req[log] ← max(req[log], seq).
4    RegisterRequest(req, c).
5  procedure RegisterReadOnlyRequest(dep)
6    RegisterRequest(dep, ⊥).
7  procedure RegisterRequest(req, c)
8    if durable_seq[i] ≥ req[i] for all shards i then
9      Ack(c) and return.
10   Insert the request into waitlists of unsatisfied shards.
11 procedure OnDurableAdvanced(i, durable)
12   durable_seq[i] ← durable.
13   Wake requests whose requirement on shard i is now
     satisfied.
14   Ack each request whose all requirements are satisfied.
```

Fig. 4 Durable acknowledgment in Ayame. A transaction is acknowledged only after its own log record and dependency frontier are durable.

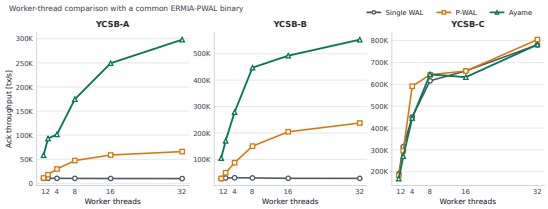

Fig. 5 Durable-acknowledgment throughput by transaction worker threads. Ayame improves update and mixed read-write workloads by combining parallel WAL logging, dependency-closed durable acknowledgment, and timestamp-integrated logical ordering.

Figure 5 shows the worker-thread comparison. The results show that Ayame improves update and mixed read-write workloads where durable logging remains on the critical path. We also evaluate YCSB-C, where all transactions are read-only and WAL records are skipped. In this case, durability work largely disappears; after using the same binary and a read-only empty-frontier fast path, Single WAL, P-WAL, and Ayame perform similarly at 32 worker threads. We therefore use YCSB-C as a sanity check for read-only bookkeeping overhead rather than as evidence of WAL persistence performance.

Ayame's timestamp integration is primarily a design simplification: it removes duplicated logical ordering between concurrency control and WAL durability by reusing ERMIA's commit timestamp as the logical recovery order.

© 1992 Information Processing Society of Japan

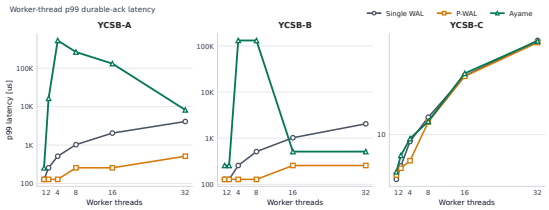

Fig. 6 P99 durable-acknowledgment latency by transaction worker threads.

Fig. 7 Pending durable commits by transaction worker threads.

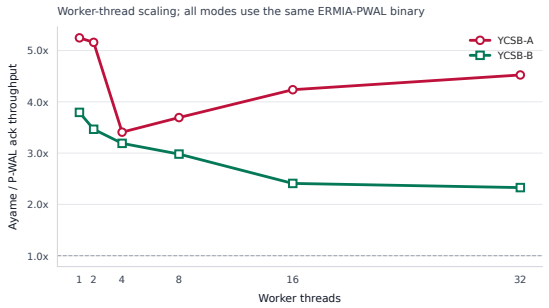

Fig. 8 Ayame throughput relative to P-WAL by transaction worker threads.

Table 1 End-to-end YCSB results at 32 transaction worker threads.

Workload	System	Ack tps	p99 μs	Pending	fdatasync	Tx/sync	Frontier B/tx	WAL atomic/tx
YCSB-A	Single WAL	9.9K	4096	0	39582	1.0	0.0	6.01
YCSB-A	P-WAL	65.9K	512	0	263401	1.0	0.0	6.00
YCSB-A	Ayame	298.1K	8192	703	82107	14.5	56.0	0.00
YCSB-B	Single WAL	28.0K	2048	0	45012	2.5	0.0	0.90
YCSB-B	P-WAL	237.6K	256	0	381791	2.5	0.0	0.90
YCSB-B	Ayame	553.2K	512	58	119651	18.5	56.0	0.00
YCSB-C	Single WAL	780.9K	41	0	0	0.0	0.0	0.00
YCSB-C	P-WAL	804.5K	40	0	0	0.0	0.0	0.00
YCSB-C	Ayame	782.0K	41	0	0	0.0	56.0	0.00

Ack tps is durable-acknowledgment throughput. Ayame uses 7 flusher threads and 1 committer thread at 32 workers.

Table 2 Ayame throughput relative to P-WAL at 32 transaction worker threads.

Workload	P-WAL tps	Ayame tps	Speedup	Ayame p99 μs	Ayame pending
YCSB-A	65.9K	298.1K	4.52×	8192	703
YCSB-B	237.6K	553.2K	2.33×	512	58
YCSB-C	804.5K	782.0K	0.97×	41	0

We do not rely on timestamp integration alone as a direct throughput optimization; the main performance effect comes from parallel logging, dependency-closed durable acknowledgment, and separating workers from durable-wait processing.

Table 1 summarizes the main 32-worker end-to-end results used in the figures. Table 2 extracts the Ayame-vs-P-WAL speedups from the same measurements.

Table 3 summarizes the 32-worker perf-stat analysis. Table 4 shows the worker-scaling perf-stat data for YCSB-B, where Ayame's throughput improvement is most visible.

Finally, Table 5 reports the top self-time symbols for YCSB-C. Since YCSB-C is read-only, this table is used as a sanity check that the remaining cost is read-path bookkeep-

Table 3 Perf-stat summary at 32 transaction worker threads.

Workload	System	Ack tps	CPU cores	Cycles/tx	Instr/tx	IPC	Ctx sw.	Tx/sync
YCSB-A	Single WAL	9.6K	1.2	316310	648780	2.06	125696	1.0
YCSB-A	P-WAL	65.2K	19.5	760326	447343	0.59	1535950	1.0
YCSB-A	Ayame	295.7K	25.7	216136	144297	0.67	2971234	14.5
YCSB-B	Single WAL	27.1K	1.1	98355	157182	1.60	136108	2.5
YCSB-B	P-WAL	246.6K	18.6	191636	108816	0.57	1423509	2.5
YCSB-B	Ayame	542.5K	31.1	145051	63165	0.44	2462420	17.5
YCSB-C	Single WAL	791.5K	40.0	131145	28270	0.22	4927	0.0
YCSB-C	P-WAL	810.8K	40.0	128225	28264	0.22	4198	0.0
YCSB-C	Ayame	772.3K	40.2	135099	32551	0.24	35684	0.0

Table 4 YCSB-B perf-stat worker scaling.

Workers	Single tps	P-WAL tps	Ayame tps	Single CPU	P-WAL CPU	Ayame CPU
1	28.9K	29.3K	122.4K	0.5	0.4	1.6
2	32.6K	53.9K	195.1K	0.6	0.9	2.9
4	32.5K	97.7K	298.0K	0.6	1.8	5.5
8	32.6K	161.8K	452.1K	0.6	3.7	10.6
16	29.5K	205.7K	510.4K	0.8	8.5	19.7
32	29.0K	247.4K	570.6K	1.0	18.1	32.2

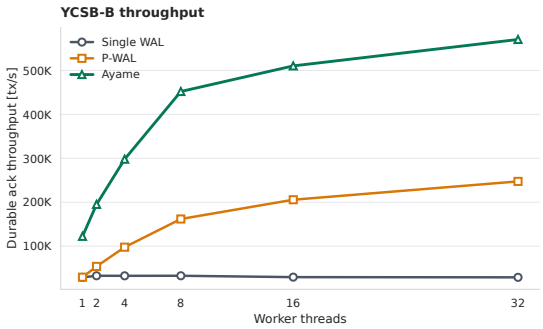

Fig. 9 YCSB-B durable-acknowledgment throughput in the perf-stat worker-scaling runs.

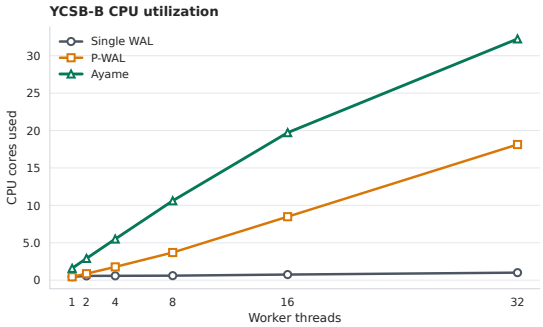

Fig. 10 CPU-core utilization in the YCSB-B perf-stat worker-scaling runs.

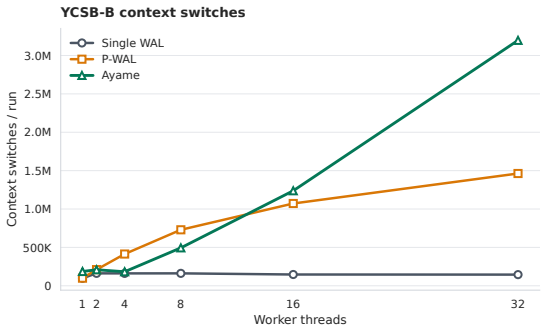

Fig. 11 Context-switch counts in the YCSB-B perf-stat worker-scaling runs.

Fig. 12 Perf-stat summary for YCSB-B worker scaling.

© 1992 Information Processing Society of Japan

Table 5 Top self-time symbols for YCSB-C at 32 transaction worker threads.

System	Rank	Self %	Symbol
Single WAL	1	58.08	TxExecutor::read
Single WAL	2	23.07	TxExecutor::ssn_parallel_commit
Single WAL	3	4.22	MasstreeWrapper¡Tuple¿::get_value
Single WAL	4	3.17	TxExecutor::mainte
Single WAL	5	2.74	TxExecutor::read_internal
P-WAL	1	58.93	TxExecutor::read
P-WAL	2	22.93	TxExecutor::ssn_parallel_commit
P-WAL	3	3.93	MasstreeWrapper¡Tuple¿::get_value
P-WAL	4	3.13	TxExecutor::mainte
P-WAL	5	2.81	TxExecutor::read_internal
Ayame	1	55.75	TxExecutor::read
Ayame	2	21.67	TxExecutor::ssn_parallel_commit
Ayame	3	4.08	MasstreeWrapper¡Tuple¿::get_value
Ayame	4	3.21	TxExecutor::read_internal
Ayame	5	3.03	TxExecutor::mainte

YCSB-C is read-only; WAL bytes and fdatasync counts are zero for all systems.

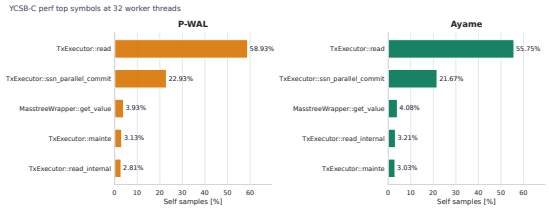

Fig. 13 Top self-time symbols for YCSB-C at 32 transaction worker threads.

ing rather than WAL persistence.

5. Related Work

6. Conclusion

Acknowledgments

References

[1] Bernstein, P. A., Hadzilacos, V. and Goodman, N.: *Concurrency Control and Recovery in Database Systems*, Addison-Wesley (1987).

[2] Tu, S., Zheng, W., Kohler, E., Liskov, B. and Madden, S.: Speedy transactions in multicore in-memory databases, *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles*, SOSP '13, New York, NY, USA, Association for Computing Machinery, p. 18–32 (online), DOI: 10.1145/2517349.2522713 (2013).

[3] Kim, K., Wang, T., Johnson, R. and Pandis, I.: ERMIA: Fast Memory-Optimized Database System for Heterogeneous Workloads, *Proceedings of the 2016 International Conference on Management of Data*, SIGMOD '16, New York, NY, USA, Association for Computing Machinery, p. 1675–1687 (online), DOI: 10.1145/2882903.2882905 (2016).

[4] Yu, X., Pavlo, A., Sanchez, D. and Devadas, S.: TicToc: Time Traveling Optimistic Concurrency Control, *Proceedings of the 2016 International Conference on Management of Data*, SIGMOD '16, New York, NY, USA, Association for Computing Machinery, p. 1629–1642 (online), DOI: 10.1145/2882903.2882935 (2016).

[5] Ports, D. R. K. and Grittner, K.: Serializable Snapshot Isolation in PostgreSQL, *Proc. VLDB Endow.*, Vol. 5, No. 12, pp. 1850–1861 (online), DOI: 10.14778/2367502.2367523 (2012).

[6] Wang, T., Johnson, R., Fekete, A. and Pandis, I.: Efficiently Making (Almost) Any Concurrency Control Mechanism Serializable, *The VLDB Journal*, Vol. 26, No. 4, pp. 537–562 (online), DOI: 10.1007/s00778-017-0463-8 (2017).

[7] Tanabe, T., Hoshino, T., Kawashima, H. and Tatebe, O.: An analysis of concurrency control protocols for in-memory databases with CCBench, *Proc. VLDB Endow.*, Vol. 13, No. 13, p. 3531–3544 (online), DOI: 10.14778/3424573.3424575 (2020).

[8] Haerder, T. and Reuter, A.: Principles of transaction-oriented database recovery, *ACM Comput. Surv.*, Vol. 15, No. 4, p. 287–317 (online), DOI: 10.1145/289.291 (1983).

[9] Mohan, C., Haderle, D., Lindsay, B., Pirahesh, H. and Schwarz, P.: ARIES: a transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging, *ACM Trans. Database Syst.*, Vol. 17, No. 1, p. 94–162 (online), DOI: 10.1145/128765.128770 (1992).

[10] Zheng, W., Tu, S., Kohler, E. and Liskov, B.: Fast Databases with Fast Durability and Recovery Through Multicore Parallelism, *Proceedings of the 11th USENIX Symposium on Operating Systems Design and Implementation*, OSDI '14, USENIX Association, pp. 465–477 (online), available from ⟨https://www.usenix.org/conference/osdi14/technical-sessions/presentation/zheng_wenting⟩ (2014).

[11] Wang, T., Johnson, R., Fekete, A. and Pandis, I.: Scalable Logging Through Emerging Non-Volatile Memory, *Proc. VLDB Endow.*, Vol. 7, No. 10, pp. 865–876 (online), available from ⟨https://www.vldb.org/pvldb/vol7/p865-wang.pdf⟩ (2014).

[12] 神谷, 孝., 川島, 英., 星野, 喬., 建部, 修., Komei, K., Hideyuki, K., Takashi, H. and Osamu, T.: P-WAL: Paralell Write-ahead Logging, **情報処理学会論文誌データベース（*TOD*）**, Vol. 10, No. 1, pp. 24–39 (online), available from ⟨https://cir.nii.ac.jp/crid/1050282812885062144⟩ (2017).

[13] Kawashima, H.: Next-generation Database Tsurugi: Concurrency Control and Recovery Systems, *IPSJ Magazine*, Vol. 66, No. 4, pp. e9–e15 (online), DOI: 10.20729/0002001646 (2025).

© 1992 Information Processing Society of Japan